

Uart_Esp32 - Projektdokumentation - Firmware

Gemeinsamer UART-Basisbetrieb und vier getrennte UART-Menues

Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART Einstellungen, Konsole, Decoder und Probe-Runner
Erstellungsdatum	2026-09-20
Autor / verantwortliche Person	OpenAI ChatGPT (technische Bearbeitung)
Version	v0.15
Status	Entwurf - lokale C++17-Syntaxpruefung und Host-Tests erfolgreich; PlatformIO-Build und Hardwaretest ausstehend
Aenderungsdatum	2026-09-20
Ablageort	Projektordner Uart_ESP32/docs
Referenzen	BX3_ESP32_Login_Leitfaden.pdf; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf; Projektstand v0.14

Kurzfassung: v0.15 ersetzt die exklusiven UART-Modi durch einen gemeinsamen RX-Datenstrom. UART Konsole und UART Decoder laufen parallel; der UART Probe-Runner besitzt ein eigenes Menue. TX wird separat und bewusst freigegeben. WLAN, MQTT, OTA, Batterie und Deep Sleep bleiben erhalten.

Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.12	2026-09-17	OpenAI ChatGPT	Entwurf	st10-Decoder mit Outer-/Inner-Frame, Commands und XOR-Prüfung.
v0.13	2026-09-20	OpenAI ChatGPT	Entwurf	Automatischer UART Probe-Runner für den Katalog mit 1000 Kandidaten.
v0.14	2026-09-20	OpenAI ChatGPT	Entwurf	BX3-Konsolenmodus mit Webterminal und bidirektionalem UART-Zugriff.
v0.15	2026-09-20	OpenAI ChatGPT	Entwurf	UART auf gemeinsamen Basisbetrieb umgestellt, vier getrennte UART-Menüs eingeführt, TX separat freigebbar gemacht, universelle Konsole erweitert und Probe-Runner-Fortschritt korrigiert.

1. Ziel und Anlass

Im Projektstand v0.14 waren **Raw/Sniffer**, **Protokoll-Decoder** und **BX3 Konsole** als gegenseitig ausschließende UART-Betriebsarten organisiert. Dadurch konnte die Webkonsole nur Daten anzeigen, wenn ausdrücklich der Konsolenmodus ausgewählt war. Befand sich die Firmware im Decoder-Modus, wurden die empfangenen Bytes zwar im allgemeinen Rohdatenpuffer gezählt und dekodiert, aber nicht in den Konsolenpuffer übernommen.

Ziel von v0.15 ist deshalb eine klare Trennung zwischen der **physischen UART-Verbindung** und den darauf aufsetzenden Funktionen. Die Bedienoberfläche erhält vier eigenständige Hauptbereiche:

- **UART Einstellungen**
- **UART Konsole**
- **UART Decoder**
- **UART Probe-Runner**

WLAN, NTP, MQTT, Batterieüberwachung, Deep Sleep, OTA und die übrige bestehende Infrastruktur bleiben erhalten.

2. Ausgangslage aus dem BX3-Leitfaden

Der beigegefügte Leitfaden beschreibt auf dem beobachteten BX3-Konsolenkanal lesbaren Linux-Boottext mit `Welcome to BX3` und `bx3 login:`. Als Arbeitsparameter werden **115200 Baud, 8N1** genannt. Für eine Anmeldung muss der ESP32 die Bytes zwischen PC/Webterminal und dem BX3-UART transparent übertragen; Zugangsdaten werden nicht durch die Firmware ersetzt.

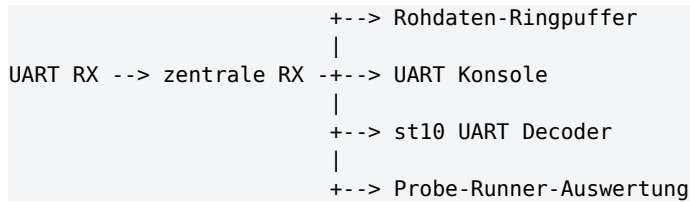
Für die elektrische Verbindung nennt der Leitfaden folgende Sicherheitsanforderungen:

- gemeinsame Masse zwischen BX3 und ESP32,
- BX3-VCC nicht mit 3,3 V oder 5 V des ESP32 verbinden,
- Logikpegel vor aktivem TX messen,
- BX3 Console-RX erst sicher identifizieren,
- ESP32-TX bei der Erprobung über 470 Ohm bis 1 kOhm Serienwiderstand anschließen.

Der physische BX3-Console-RX-Punkt ist durch die vorliegenden Unterlagen weiterhin nicht eindeutig bestimmt. Das ist eine Hardwarefrage und wird durch die Firmwareänderung nicht gelöst.

3. Neue UART-Architektur ab v0.15

Seit v0.15 existiert nur noch **ein gemeinsamer UART-Basisbetrieb**. Ein empfangenes Byte wird gleichzeitig an alle passiven Verbraucher verteilt:



Damit ist die Konsole nicht mehr davon abhängig, welcher Decoderbereich geöffnet ist. Solange der UART läuft, kann derselbe Datenstrom gleichzeitig im Terminal dargestellt und vom st10-Decoder ausgewertet werden.

Für TX gibt es eine zentrale Freigabe und eine Senderreservierung:



Der Probe-Runner und der Feld-5-Replay reservieren TX während einer aktiven Sendefolge. In dieser Zeit bleibt RX in der Konsole sichtbar, manuelle Konsoleneingaben werden jedoch blockiert.

4. Menüstruktur

Die Startseite enthält weiterhin die vorhandenen Infrastrukturmenüs und zusätzlich vier klar getrennte UART-Menüs.

Menü	Aufgabe
UART Einstellungen	Gemeinsame Hardware-UART-Konfiguration, Start/Stop und separate TX-Freigabe.
UART Konsole	Universelles serielles Live-Terminal für RX und manuelle TX-Eingaben.
UART Decoder	st10-Protokollauswertung, Raw-Daten, Felder, Kalibrierung und Feld-5-Replay.
UART Probe-Runner	Aktiver automatischer Test der 1000 Kandidaten mit Fortschritts- und Reaktionsanzeige.
WLAN / NTP	Bestehende Netzwerk- und Zeitkonfiguration, unverändert erhalten.
MQTT	Bestehende MQTT-Konfiguration, ergänzt um UART-TX-Statuswerte.
Batterie / ADC	Bestehende Batteriefunktion, unverändert erhalten.
Deep Sleep	Bestehende Deep-Sleep-Funktionen, unverändert erhalten.
OTA Update	Bestehende OTA-Funktionen, unverändert erhalten.

5. UART Einstellungen

Die Seite `/uart/settings` ist jetzt ausschließlich für die gemeinsame UART-Verbindung zuständig. Gespeichert werden:

- UART-Basisbetrieb aktiviert/deaktiviert,
- Hardware-UART 1 oder 2,
- RX-GPIO,
- konfigurierter TX-GPIO,

- **TX-Ausgang separat freigegeben/gesperrt**,
- Baudrate,
- Frameformat 8N1, 8E1, 8O1 oder 8N2.

5.1 Separate TX-Freigabe

Ein eingetragener TX-GPIO bedeutet nicht automatisch, dass der ESP32 die Leitung treibt. Ist TX gesperrt, startet die Firmware HardwareSerial mit TX=-1. Dadurch wird der konfigurierte GPIO nicht vom UART-Peripherieblock als TX-Ausgang verwendet.

Das ermöglicht einen sicheren passiven Start:

```
RX: EIN
TX: AUS
```

Erst nach Prüfung von Ziel-RX und Pegeln kann TX bewusst aktiviert werden:

```
RX: EIN
TX: EIN
```

5.2 Migration älterer Einstellungen

Die früher gespeicherte NVS-Betriebsart wird beim Upgrade übernommen: War zuvor ein UART-Modus aktiv, wird der neue UART-Basisbetrieb aktiviert. **TX wird bei der Migration absichtlich nicht automatisch freigegeben.** Der Anwender muss TX einmal bewusst unter UART Einstellungen aktivieren.

Zur Abwärtskompatibilität bleibt der alte NVS-Schlüssel mode als einfacher Ein/Aus-Marker erhalten.

6. UART Konsole

Die Seite /uart/console ist ab v0.15 eine **universelle UART-Konsole** und nicht mehr an BX3 als exklusiven Modus gebunden.

6.1 Empfang und Darstellung

Jedes empfangene UART-Byte wird in einen eigenen **8192-Byte-Ringpuffer** kopiert. Der Browser fragt neue Bytes inkrementell über eine Sequenznummer ab. Die Darstellung kann zwischen folgenden Ansichten umgeschaltet werden:

- Text,
- HEX,
- HEX + ASCII.

Der Browser pollt die Konsole derzeit alle 150 ms. Bei einem überholten Browser-Stand meldet die API, dass ältere Daten bereits aus dem Ringpuffer gefallen sind.

6.2 Senden

Für manuelle TX-Ausgaben stehen zur Verfügung:

- Text / ASCII,
- frei eingebbare HEX-Bytes, zum Beispiel 48 65 6C 6C 6F 0D,
- Zeilenabschluss CR, LF, CRLF oder keiner,
- Enter,
- Ctrl+C,
- Ctrl+D,
- TAB,
- ESC,
- optional verdeckte Texteingabe für Passwörter.

Eine Konsoleneingabe wird nur gesendet, wenn UART läuft, TX separat freigegeben ist, ein gültiger TX-GPIO gesetzt ist und TX nicht gerade vom Probe-Runner oder Feld-5-Replay reserviert wird.

Hinweis: Das Webinterface besitzt keine eigene TLS- oder Login-Schicht. Zugangsdaten und Shell-Sitzungen sollten deshalb nur in einem vertrauenswürdigen Netz verwendet werden.

7. UART Decoder

Die Seite /uart/decoder verarbeitet den gemeinsamen RX-Datenstrom permanent parallel zur Konsole. Der st10-Decoder bleibt inhaltlich erhalten:

- Synchronisation auf äußerem Header FF FB,

- Outer- und Inner-Längenprüfung,
- Inner-Frame FF FD/FE,
- XOR-Prüfung,
- bekannte Commands 0x0021, 0x0023, 0x0031, 0x0033,
- Feldwerte und normierte Werte,
- Aliasnamen,
- Totzone und Kalibrierwerte,
- Raw-HEX/ASCII-Anzeige.

Aliasnamen, Kalibrierung und Totzone werden jetzt über die eigene Route /save_uart_decoder gespeichert. Dadurch muss eine reine Decoder-Darstellungsänderung den Hardware-UART nicht neu konfigurieren.

7.1 Feld-5-Replay

Der experimentelle Feld-5-Replay bleibt erhalten. Er benötigt:

- laufenden UART,
- explizite TX-Freigabe,
- gültigen TX-GPIO,
- ein frisches gültiges 0x0021-Paket,
- keinen gleichzeitig laufenden Probe-Runner.

Während der Replay aktiv sendet, ist manuelles Konsolen-TX blockiert.

8. UART Probe-Runner

Der Probe-Runner besitzt ab v0.15 eine eigene Hauptseite /uart/probe. Er ist kein Unterbereich des Decoders mehr.

Die bestehende Testlogik bleibt erhalten:

- 2 Sekunden Baseline,
- 1000 Katalogkandidaten,
- je Kandidat 5 Sekunden Testzeit,
- Senden alle 250 ms,
- Reihenfolge P0 -> P1 -> P2 -> P3,
- optional bei erster erkannten Reaktion stoppen oder alle 1000 testen.

Während des Probe-Runs läuft der gemeinsame RX-Pfad weiter. Konsole und Decoder können die Antworten deshalb weiterhin anzeigen und auswerten. TX ist in dieser Phase exklusiv für den Probe-Runner reserviert.

8.1 Korrektur des Kandidatenfortschritts

Im vorherigen Stand wurde `_probeOrderIndex` beim Abschluss eines Kandidaten zweimal erhöht. Dadurch konnte jede zweite Katalogposition übersprungen werden. In v0.15 existiert an dieser Stelle nur noch **eine** Erhöhung des Indexes.

Der Quellcode wurde statisch darauf geprüft, dass genau eine `++_probeOrderIndex`-Operation vorhanden ist.

9. TX-Zustand und Senderreservierung

Die Firmware stellt den aktuellen TX-Zustand zentral als Text bereit. Mögliche Zustände sind unter anderem:

Zustand	Bedeutung
gesperrt	TX ist in UART Einstellungen nicht freigegeben.
UART gestoppt	Hardware-UART läuft nicht.
kein gueltiger TX GPIO	TX ist freigegeben, aber der konfigurierte GPIO ist nicht nutzbar.
frei	Manuelles Konsolen-TX ist möglich.
UART Probe-Runner	Probe-Runner besitzt aktuell die TX-Schnittstelle.
Decoder Feld-5-Replay	Experimenteller Feld-5-Replay besitzt aktuell TX.

Dieser Zustand wird sowohl in der Weboberfläche als auch im UART-Status-JSON verwendet.

10. Webrouten und APIs

Route	Funktion
/uart/settings	Gemeinsame UART-Hardwareeinstellungen.
/save_uart	Speichert Basisbetrieb, Hardwareparameter und TX-Freigabe.
/uart/console	Universelles UART-Terminal.
/uart/console/data	Inkrementelle Konsolendaten als JSON mit Text- und HEX-Darstellung.
/uart/console/send	Sendet Text oder HEX-Bytes, wenn TX verfügbar ist.
/uart/console/control	Sendet definierte Steuerzeichen.
/uart/console/clear	Leert den Konsolen-Ringpuffer.
/uart/decoder	st10-Decoder-Seite.
/save_uart_decoder	Speichert Decoder-Aliase, Kalibrierung und Totzone.
/uart/probe	Probe-Runner-Seite.
/uart/probe/start_hit	Start mit Stop bei erkannter Reaktion.
/uart/probe/start_all	Start für vollständigen Katalogdurchlauf.
/uart/probe/stop	Stoppt den Probe-Runner.
/uart/status	Gemeinsamer Live-Status für UART, Decoder und Probe-Runner.
/uart/start //uart/stop	Startet bzw. stoppt den Hardware-UART.
/uart/monitor	Kompatibilitäts-Redirect auf /uart/decoder.
/uart	Kompatibilitäts-Redirect auf /uart/decoder.

11. MQTT-Erweiterung

Die bisherigen UART-MQTT-Werte bleiben erhalten. Zusätzlich veröffentlicht v0.15:

- .../uart/enabled
- .../uart/tx_enabled
- .../uart/tx_owner

Der bestehende Topic .../uart/mode bleibt aus Kompatibilitätsgründen bestehen und liefert nun UART Basisbetrieb beziehungsweise Aus.

12. Erhaltene Projektfunktionen

Die Änderung ist auf die UART-Struktur beschränkt. Folgende bestehende Funktionen wurden nicht entfernt:

- WLAN-Client und WLAN-Fallback/Provisionierung,
- NTP und CET/CEST-Zeitlogik,
- Webinterface,
- MQTT,
- Deep Sleep,
- Batterie / ADC,
- Browser OTA,
- ArduinoOTA / PlatformIO OTA,
- HTTP Pull OTA,

- NVS / Preferences,
- interne ESP32-Temperatur,
- Web-Reboot.

Die vorhandenen WLAN- und sonstigen gespeicherten Konfigurationen werden nicht durch die UART-Migration zurückgesetzt.

13. Geänderte Dateien

Datei	Änderung
src/UartManager.h	Exklusive UART-Modi entfernt; Basisbetrieb, TX-Freigabe, TX-Owner und 8192-Byte-Konsole ergänzt.
src/UartManager.cpp	Gemeinsamer RX-Pfad, TX-Gating, NVS-Migration, paralleler Decoder/Konsole, Probe-Fortschrittsfix und erweiterte JSON-Statuswerte.
src/main.cpp	Vier UART-Menüs, universelle Konsole, separate Decoder-/Probe-Seiten, neue Routen und MQTT-UART-Statuswerte.
src/ConfigDefaults.h	Firmwareversion auf 0.15.0 angehoben.
README.md	Architektur und Bedienung für v0.15 dokumentiert.
VALIDATION_v0.15.txt	Validierungsstand, Prüfungen und offene Hardwaretests dokumentiert.
docs/2026-09-20 - Uart_Esp32 - Projektdokumentation - Firmware - v0.15.*	PDF-, DOCX- und Markdown-Dokumentation.
docs/2026-09-20 - Uart_Esp32 - Commit-Nachricht - UART-Architektur - v0.15.txt	Vorgeschriebene englische Commit-Nachricht.

14. Validierung

Folgende Prüfungen wurden lokal erfolgreich durchgeführt:

1. C++17-Syntaxprüfung von UartManager.cpp und TestCandidateCatalog.cpp gegen lokale Arduino/ESP32-Schnittstellen-Stubs.
2. C++17-Syntaxprüfung von main.cpp gegen lokale Framework-Stubs.
3. Host-Test für Konsolen-JSON, HEX-Ausgabe, TX-Gating, TX-Reservierung und Ringpuffer-Überlauf.
4. Statische Prüfung der vier UART-Routen und der weiterhin vorhandenen Menüs für WLAN, MQTT, Batterie, Deep Sleep und OTA.
5. Statische Prüfung, dass der Probe-Runner-Index nur noch einmal erhöht wird.
6. Prüfung, dass keine alten UartManager::Mode, setMode oder Mode::-Abhängigkeiten im UART-Quellcode verblieben sind.

Nicht durchgeführt: In der Arbeitsumgebung ist PlatformIO nicht installiert. Deshalb konnte kein echter ESP32-Arduino-Framework-Build erzeugt und keine Firmware auf Hardware geflasht werden. Ebenso wurde kein realer Browser gegen einen laufenden ESP32 und kein elektrischer BX3-UART-Test durchgeführt.

15. Grenzen der universellen UART-Konsole

„Universell“ bezieht sich auf die Firmwarefunktion für einen ESP32-TTL-UART innerhalb der unterstützten Hardwareparameter. Die Konsole ersetzt keine elektrische Schnittstellenanpassung.

Nicht direkt abgedeckt sind beispielsweise:

- RS-232-Pegel,
- RS-485 ohne externen Transceiver und Richtungssteuerung,
- 5-V-Signale außerhalb der zulässigen ESP32-Pegel,
- zusätzliche Hardware-Flow-Control-Leitungen wie RTS/CTS,

- UART-Formate außerhalb der aktuell angebotenen 8N1, 8E1, 8O1, 8N2.

Für kontinuierliche sehr hohe Datenraten verwendet die Webkonsole weiterhin HTTP-Polling und keinen WebSocket. Der ESP32-Ringpuffer wurde auf 8192 Bytes vergrößert, kann bei dauerhaft höherem Datenaufkommen als Browser und Netzwerk verarbeiten dennoch ältere Daten überschreiben. Dieser Zustand wird der Browser-API als truncated gemeldet.

16. Inbetriebnahme-Checkliste v0.15

- ☐ Projekt mit PlatformIO für uart_esp32_usb erfolgreich bauen.
- ☐ Firmware flashen und Startseite öffnen.
- ☐ WLAN / NTP, MQTT, Batterie, Deep Sleep und OTA auf vorhandene Einstellungen prüfen.
- ☐ Unter UART Einstellungen zunächst TX gesperrt lassen.
- ☐ UART-Port, RX-GPIO, Baudrate und Frameformat prüfen.
- ☐ UART starten und kontrollieren, dass RX gleichzeitig in UART Konsole und UART Decoder sichtbar ist.
- ☐ Vor aktivem TX Zielpiegel messen und Ziel-RX sicher identifizieren.
- ☐ TX bewusst freigeben und zunächst nur einen kontrollierten Test senden.
- ☐ Beim BX3 den im Leitfaden empfohlenen Serienwiderstand in der ESP32-TX-Leitung verwenden.
- ☐ Probe-Runner nur an einem autorisierten und betriebssicheren Testaufbau verwenden.

17. Quellenbasis

1. **BX3 / ESP32-S3 - Technischer Leitfaden für Konsolenzugriff, Login und Firmware-Bridge**, Stand September 2026. Verwendet wurden insbesondere die Angaben zu bx3 login:, 115200/8N1, transparenter serieller Bridge, Verdrahtung, Pegelprüfung und TX-Schutzwiderstand.
2. **Projektdokumentation - Regeln**, Stand 2026-06-18. Verwendet wurden insbesondere die Vorgaben zu Dateinamen, Versionsnummer, Metadaten, Änderungshistorie, Codepaketstruktur, englischer BL_/BS_-Commit-Nachricht sowie PDF- und DOCX-Auslieferung.
3. Projektquellstand **Uart_Esp32 v0.14** als technische Ausgangsbasis der Firmwareänderung.